CHAPTER 8: PROMPTS AS ARCHITECTURE

Summary: *A prompt is not a message. It is a durable component with a version, a cache key, an ordering dependency and a failure mode, and treating it as text you tweak is how teams end up with instructions that contradict each other and nobody can say which one wins. This chapter covers what actually makes a model follow a rule, what caching charges and what silently destroys it, and how reusable prompt assets are packaged so they can be governed rather than copied.*

A Contradiction Must Be Removed, Not Arbitrated

Start with the failure that produces the most wrong answers, because the wrong answers are all reasonable-sounding.

A prompt contains two instructions that cannot both be honoured. Somebody notices. The available responses look like a menu: state which one takes precedence, move one of them to a different position, put one in bold, lower the temperature so behaviour is more consistent, add a note explaining when each applies.

Only one of those is a **fix**, and it is the one that **deletes the contradiction** from the prompt. Everything else leaves both directives live and merely changes which one tends to win, or how often, or how visibly. A tie-break rule is not a stable, documented resolution the way a rewritten instruction is, because the model is still being asked to reconcile two things you could have reconciled yourself. Lowering the temperature does not resolve a conflict, it narrows the distribution of which conflict resolution you get.

The **named anti-pattern is arbitrating a contradiction instead of removing** it. It survives review because it looks like added rigour.

Name the underlying condition: the **underspecification gap** is the distance between what you told the model and what you actually require, and every symptom in this chapter lives in it. A contradiction is that gap made visible. Most of the gap is not visible at all.

The documented golden rule is the cheapest test available for it: a prompt that would **confuse a colleague with minimal context** will **confuse Claude**.[1] If a competent human handed only that text would have to come back with a clarifying question, the model will not come back.

Position and Separation Are the Levers for Long Prompts

When a rule sits in a long prompt and is not being followed, there are four plausible remedies and only one is a documented lever.

Where the rule **sits**, and how clearly it is **fenced off from the reference material** around it. That is the lever: position plus structural separation.

Repeating the rule more often is not that lever. Typesetting it differently is not either. Changing the sampling temperature acts on how tokens are chosen, not on how the instruction is read. Each of the three is doing something real and none of them addresses the actual problem, which is that an instruction is sitting undifferentiated in a wall of material that also contains inputs.

That is why examples get wrapped in example tags: so the model can tell an example from an instruction.[1] The same instinct applies to everything else in a long prompt. The model has to be able to **tell an instruction from an input**, and the only thing that tells it is **structure**.

Adherence Improves Along Two Independent Axes

When a scenario asks what would improve rule-following, and expects more than one answer, the two correct ones sit on separate axes.

Spell the rules out with **no room for reading**, and say **which of them beats the others**. This attacks the underspecification gap directly.

Show what compliance looks like on the specific cases currently failing. Not generic examples. Examples of the failure, done correctly.

Those are independent, which is exactly why both get selected: one removes ambiguity about the rule, the other removes ambiguity about its application.

Options that *remove* structure or *add* variance go the other way, whatever they promise.

What Newer Models Changed, and Why Old Prompts Misbehave

Three documented behavioural changes will produce a scenario where a prompt that used to work now does not.

Increased responsiveness to the system prompt. Claude Opus 4.5 and Opus 4.6 respond more strongly to system prompt instructions, and the documented consequence is direct: prompts written to stop a tool being *under*triggered may now cause it to *over*trigger, and forceful phrasing such as "CRITICAL: You MUST use this tool when…" should be dialled back to ordinary language.[1] A prompt is **calibrated against a model**, and the **calibration does not transfer**.

Response length behaves differently per model. For Claude Sonnet 5, length is calibrated to task complexity rather than to a fixed verbosity.[2] For Claude Opus 5, default user-facing responses run longer than previous Opus models, and here is the part that catches people: **effort** controls **how much the model thinks**, not **how much it says**, so turning effort down does not reliably shorten the visible answer. Length has to be prompted for explicitly.[3]

A **capability upgrade is a reason to re-examine your scaffolding**. Claude Fable 5 and Claude Mythos 5 differ behaviourally from Claude Opus 4.8 in ways that may need prompt or scaffolding updates, and a capability improvement at that level is itself a reason to ask which instructions, tools and guardrails are still earning their place.[4] Guardrails written to compensate for a weakness that no longer exists are now just tokens and constraint.

Prefill: Recommended on One Page, Rejected by the API

This one deserves its own section because the documentation genuinely disagrees with itself, and a scenario can be built out of that disagreement.

Techniques listed for increasing output consistency include prefilling the assistant turn to skip Claude's preamble and force a structure.[5] Guidance on reducing

prompt leak also lists prefilling the assistant turn as a way to re-emphasise key instructions.[6]

And: **prefilled responses on the last assistant turn** are **no longer supported** starting with **Claude 4.6 models** and **Claude Mythos Preview**, and such requests **return a 400 error**.[1]

Both statements are published. The resolution is the one you would apply to any two conflicting sources: the **constraint that the API enforces** wins over the **technique a page recommends**. If a scenario names a 4.6-or-later model, prefill is not available regardless of which page suggested it.

Two related notes on guardrail cost. Leak-resistant prompt engineering should be used **only when necessary**, because the added complexity can degrade performance on the rest of the task.[6] And the prompt engineering guidance assumes you already have success criteria and a way to evaluate against them, while stating plainly that **not every failing evaluation is best solved by prompt engineering**.[7] Some failures are data problems, retrieval problems or architecture problems wearing a prompt costume.

Structured Outputs Constrain the Shape, Not the Capitalisation

Structured outputs constrain a response to a schema using constrained decoding. One documented gap will bite a design that assumes the schema is absolute.

String enum and const values are **not guaranteed to match** the schema's **capitalisation**. The response completes normally. There is no error and no special stop reason.[8] So the mismatch reaches your code looking like success.

Two consequences follow. **Compare enum values case-insensitively**, and **never define two values that differ only in capitalisation**, because you have no way to distinguish them.

There is a second gap worth knowing: several SDKs transform schemas containing unsupported features, stripping numeric and length constraints out of the schema that actually gets sent, moving them into the description, and validating the response against the original constraint in the caller's own code.[8] A **constraint you**

wrote may be **enforced by your client** rather than by **decoding**, which changes where the failure surfaces.

Untrusted Content Has a Delivery Channel, and Instructions Do Not

Prompt injection defences are structural rather than persuasive, and the structure is specific.[9]

Deliver untrusted third-party content only inside tool result blocks. Never in the system prompt, never in a plain user text block. Claude is trained to treat instructions appearing inside tool results with appropriate scepticism, and that training is the defence you are trying to invoke.

The tool's own **description** should state **what this content is** and **which system it arrived** from. Then state in the system prompt that content returned from **tools, documents or searches** is **untrusted data** that must **never override the system prompt** or the user's request.

JSON-encode untrusted strings, so an attacker cannot break out of the payload and into instruction context.

Do **not put your own instructions in tool results**. This is the counter-intuitive one and it follows directly from everything above: content in a tool result is treated as untrusted data, so your instruction there may be ignored or flagged as an injection attempt. You built the sceptical channel; do not then try to give orders through it.

Screen tool output with a **lightweight model** such as **Claude Haiku 4.5**, using **structured outputs**, so the verdict comes back as a parseable value your application can branch on rather than prose you have to interpret.

Prompt Caching: Order, Position, and What Destroys Everything

Caching is where the reuse objective becomes precise, and nearly every item turns on the same property.

Cache prefixes are created in a **fixed order**: **tools**, then **system**, then **messages**. A change at one level invalidates that level and everything after it.[10] That single

sentence has a large consequence: **modifying tool definitions invalidates the entire cache**, because tools come first.

What gets hashed for the **cache key** is the **whole run of content up to the breakpoint**, **accumulated**. Position is therefore the deciding property in most scenarios: put something that **changes at the very front** and **nothing behind it can be cached** at all, no matter how large, how static, or how otherwise eligible it is. A timestamp at the top of a system prompt is not a small problem, it is a total one.

The rest of the mechanics, which scenarios are built from:

Behaviour	**The detail that matters**
Where writes happen	Only at a breakpoint
How a read works	Walks backward from the breakpoint looking for entries earlier requests wrote, checking at **most 20 positions**
The breakpoint trap	A breakpoint on a block that changes every request finds nothing. Automatic caching hits the same trap when the last cacheable block is the varying one
Match requirement	**100% identical** segments up to the breakpoint
Lifetime	**5 minutes by default**, refreshed at no extra cost each time it is used; **1 hour available at twice the base input token price**
Breakpoints	Up to **4**, and they add no cost themselves, since charging is for what is written and read

Behaviour	The detail that matters
Output tokens	**Caching has no effect on output token generation**
Isolation	Between organisations, and on the Claude API between workspaces

Two of those rows answer whole items. **Caching does nothing for output cost**, so a scenario dominated by long generated responses is not a caching problem. And the **1-hour lifetime costs twice the base input rate to write**, so it is worth it when the gap between requests is longer than five minutes and the prefix is large, and wasteful otherwise.

Worth separating two different ways to reduce spend, because a scenario may ask for both. **Caching lowers the price of the repeated portion**. **Reducing what you send shrinks the amount** at all. Those are the two levers that change what a request costs. Anything that adds tokens, or merely relocates them within the request, changes nothing.

Diagnosing a Cache Miss

Without diagnostics, the only signal that a cache unexpectedly missed is the **cache read token count dropping to zero**, with no indication of what changed.[11] That is a genuinely bad debugging position, and it is why the diagnostic path exists.

Passing the previous response id makes the API compare the two requests and report the **first point of divergence** among the model, the system prompt, the tools and the message history. The division of labour between two fields is worth memorising exactly: the **diagnostics field** answers whether the **request changed**. The **usage field** answers whether the **cache hit**.

The documented divergence types are effectively a checklist of causes:

- The **model changed**.

- The **system prompt carried an interpolated timestamp or request id**. The fix is to make the system prompt byte-stable and move dynamic data into the first user message after the breakpoint.
- The **tools array was added to, reordered, or serialised non-deterministically**. Non-deterministic serialisation is the nastiest of these because nothing in your code changed.
- The **message history was altered or truncated rather than appended** to. Caching assumes append.

Skills: Versioned, Pinned, and Loaded When Needed

The packaging question has a precise answer. A **reusable prompt asset** carries a **version owned in one place**, and it **enters the context only for the tasks** that need it. Those two properties together are what separate it from the three alternatives that keep turning up: something permanently resident, something with no version at all, and something every human on the team keeps a private copy of.

When a scenario asks for both halves of one mechanism, they are a **single centrally owned packaged artifact**, and **every workflow using that same artifact**. Adding a third option based on copying or retrieval reintroduces exactly the divergence the pairing eliminates.

Agent Skills package instructions, scripts and resources as versioned units, and the operational details are where the items live.[12]

Pin a specific version in production. Omitting the version, or setting it to latest, means a version uploaded by anyone in the workspace immediately changes what deployed agents run.

A **new version is a complete snapshot, not a delta**. The full file set must be uploaded each time, and files you leave out are not carried over from the previous version. That is a data-loss shape, not a versioning inconvenience.

Skills render into the system prompt in a fixed order, so changing the Skills list or its order breaks the prompt cache. With an unpinned version, publishing a new one

can invalidate the cached prefix if it changes the Skill's description. **Skill management and cache behaviour are therefore the same subject**, which is not obvious until it costs you.

Where the **Compliance API** is enabled, its **activity feed records creation and deletion** of **Skills and Skill versions**. Operations that happen while it is off are not recorded and cannot be recovered later. An audit trail that was switched off has no retrospective mode.

How Wrong Answers Are Built on Prompt and Reuse Questions

- **Managing a contradiction instead of deleting** it. The option adds precedence, emphasis, repetition or a lower temperature. The test: after this change, are **both instructions still in the prompt**?
- **Reaching for frequency or typography when the lever is structure**. The test: does this change where the **rule sits** or how it is **separated** from surrounding material?
- **Cache reasoning that ignores order and position**. The option caches a large static block that sits after something variable, or edits tools and expects the system cache to survive. The test: what is the **earliest thing in this request that changes**?
- A **technique applied to a model that rejects** it. The test: does the scenario **name a model version**, and does that version **still support** this?
- **Assuming the schema is absolute**. The test: what does the model return that the **schema does not actually guarantee**?

When the scenario says	The answer is usually	The trap is usually
Two instructions in a prompt conflict	Rewrite the prompt so only one instruction exists	Declaring which one wins, reordering, emphasising, or lowering temperature

When the scenario says	The answer is usually	The trap is usually
A rule in a long prompt is ignored	Change where it sits and how it is separated from reference material	Repeating it, bolding it, or adjusting sampling
Rule-following must improve, more than one answer wanted	Spell out the rules and which beats which, and show correct handling of the cases failing now	Anything that removes structure or adds variance
A tool now fires too often after a model upgrade	Soften the forceful phrasing written for the older model	Adding more emphasis, which is what caused it
Responses are too long on Opus 5	Ask for the length explicitly	Lowering effort, which controls thinking rather than speaking
A prefill is proposed on a 4.6 or later model	Not available; the request returns a 400	Following the page that recommends prefilling
An enum value comes back with unexpected capitalisation	Compare case-insensitively; never define values differing only in case	Treating the schema as a guarantee of exact casing
Untrusted third-party content enters the system	Deliver it inside tool result blocks only, JSON-encoded, and declare it untrusted in the system prompt	Putting it in the system prompt or a user text block, or putting your instructions into tool results

When the scenario says	The answer is usually	The trap is usually
Tool definitions were edited	The whole cache is invalidated; tools sit first	Assuming only the tools portion is affected
A timestamp or request id sits in the system prompt	Make the system prompt byte-stable; move dynamic values after the breakpoint	Caching the large static content that follows it
Cost is dominated by long generated answers	Caching will not help; it does not affect output tokens	Adding breakpoints
Requests recur more than five minutes apart with a large prefix	The 1-hour lifetime, at twice base input price for writes	Assuming the default 5-minute window covers it
A cache is missing and nobody knows why	Pass the previous response id and read the first divergence	Guessing, with cache read tokens at zero as the only clue
A prompt asset must not drift between teams	One centrally versioned artifact, pinned, loaded when the task needs it	Copies, retrieval-based reconstruction, or an unpinned version
A new Skill version is published	Upload the complete file set; omissions are not carried over	Treating a version as a delta

The habit worth carrying: read the request from **position zero forward** and find the **first thing that changes**. That one position explains most caching behaviour, and most caching wrong answers ignore it.

What to Remember

- A contradiction in a prompt is **removed, never arbitrated**. Precedence rules, reordering, emphasis and lower temperature all **leave both instructions live** and only change which one tends to win.
- The **underspecification gap** is the distance between what you **told the model** and what you **actually require**. The cheapest test for it: a prompt that would **confuse a colleague with minimal context** will confuse Claude, and unlike the colleague, the **model will not ask**.
- For a rule ignored inside a long prompt, the documented lever is **position plus structural separation**: where the rule sits, and how clearly it is **fenced off from reference material**. Repetition, typography and sampling changes are **not levers**. Examples go in example tags so the model can **tell an example from an instruction**.
- Adherence improves along **two independent axes**: **spelling the rules out** with no room for reading and **saying which beats** which, and **showing correct handling** of the cases that are currently failing. Both get chosen because they **fix different ambiguities**.
- **Claude Opus 4.5 and 4.6** respond **more strongly to the system prompt**, so wording written to stop *under*triggering can cause *over*triggering, and forceful phrasing should be **dialled back to ordinary language**.
- On Claude Opus 5, **effort** controls **how much the model thinks**, not **how much it says**, so lowering effort does **not reliably shorten a response**. **Ask for length** explicitly. On Sonnet 5, length is **calibrated to task complexity** rather than fixed verbosity.
- A capability upgrade is a reason to **re-examine instructions, tools and guardrails**, since some exist to **compensate for weaknesses that no longer exist**.
- **Prefilling the last assistant turn is unsupported** from Claude 4.6 and Mythos Preview and **returns a 400**, even though two other pages

recommend prefilling. An **enforced API constraint outranks** a recommended technique.

- **Leak-resistant prompt engineering is used only when necessary**, because the complexity **degrades the rest of the task**. And not every failing evaluation is **best solved by prompt engineering**.

- Structured outputs do **not guarantee the capitalisation** of string enum and const values, and the response **completes with no error** and no special stop reason. **Compare case-insensitively** and **never define values differing only in case**. Some SDKs **strip numeric and length constraints** from the sent schema and validate them in your own code instead.

- Untrusted content is delivered **only inside tool result blocks, JSON-encoded**, with the system prompt declaring that tool, document and search content must **never override** it. **Never put your own instructions in tool results**, because content there is treated as untrusted and may be ignored or flagged. **Screen tool output with a lightweight model** using structured outputs so the verdict is parseable.

- Cache prefixes are built **tools, then system, then messages**, and a change at one level **invalidates it and everything after**, so **editing tool definitions invalidates the entire cache**. The key hashes the accumulated content up to the breakpoint, so anything that changes at the **very front** destroys all caching behind it.

- A read walks backward from the breakpoint over at **most 20 positions**; hits need **100% identical segments**; the default lifetime is **5 minutes**, refreshed free on use, with **1 hour** available at **twice the base input price**; up to **4 breakpoints**, which **cost nothing themselves**; and caching has **no effect on output tokens**.

- Without diagnostics the only sign of a cache miss is **cache read tokens dropping to zero**. Passing the previous response id reports the **first divergence** among model, system prompt, tools and message history. The **diagnostics field** says whether the **request changed**; the **usage field** says

whether the **cache hit**. Common causes: a changed model, an **interpolated timestamp or request id** in the system prompt, a tools array **reordered or serialised non-deterministically**, and message history **altered or truncated instead of appended** to.

- A reusable prompt asset has **one centrally owned version** and enters the context **only for the tasks that need** it, which rules out always-resident, never-versioned and per-engineer copies. **Pin the version in production**, because unpinned or latest means **anyone's upload changes what deployed agents run**.

- A new Skill version is a **complete snapshot, not a delta**: upload the full file set, because **omitted files are not carried** over. Skills render into the system prompt in a **fixed order**, so changing the list or its order **breaks the prompt cache**. Where the Compliance API is off, Skill operations are **not recorded and cannot be recovered**.